

Zadatak 3

Ova zadaća pokriva više razina apstrakcije GPU programiranja: od najniže razine apstrakcije - implementacije kernela (CUDA ili OpenCL) do rada s alatima više razine poput Python Numbe. Preporučuje se korištenje CUDA-e ako je dostupan NVIDIA GPU, dok studenti bez takvog hardvera mogu koristiti OpenCL (*vidjeti [upute](#)*).

Rješavanje zadataka podrazumijeva izradu **programskog koda**, provedbu zadanih **mjerjenja** te tablično **bilježenje rezultata**. Također, potrebno je u kratkim crtama objasniti razlike u performansama, posljedice odabranog modela paralelizacije te procijeniti kada ima smisla koristiti alate više razine apstrakcije, a kada ne.

Zadatak 3.1

Napišite CUDA ili OpenCL kernel koji izračunava koliko ima **prim brojeva** u zadanom ulaznom nizu cjelobrojnih vrijednosti. Koristite ulazne veličine oblika $N = 2^k$, pri čemu N treba biti dovoljno velik da je vrijeme izvođenja traje nekoliko sekundi. Ulazno polje može sadržavati slučajno generirane pozitivne cijele brojeve ili vrijednosti $1, 2, \dots, N$. Implementacija mora podržavati slučaj u kojem je $G < N$, tj. jedna dretva mora moći obraditi više elemenata.

Potrebno je implementirati dvije verzije:

1. verziju **bez atomičke operacije**, u kojoj se koristi obični inkrement globalnog brojača
2. verziju **s atomičkom operacijom** za povećavanje brojača.

Izmjerite vrijeme izvođenja za različite vrijednosti:

- veličina ulaza: $N = 2^{20}, 2^{22}, 2^{24}$
- veličina bloka dretvi / radne grupe: $L = 32, 64, 128, 256, 512$
- ukupan broj dretvi: $G = B \times L$, gdje je $B = 16, 32, 64, 128, 256$.

Svako mjerenje ponovite više puta i zapišite rezultate u tablici.

U analizi objasnite kako veličina bloka/radne grupe L utječe na performanse, kako ukupan broj dretvi G utječe na performanse, što se događa kada je G znatno manji od N , te zašto povećavanje G u nekom trenutku više ne donosi ubrzanje. Posebno komentirajte razliku između atomičke i neatomičke verzije: točnost rezultata, razliku u vremenu izvođenja i razloge za opaženo ponašanje. Također provjerite mijenja li se najbolja konfiguracija ovisno o veličini ulaza N .

Zadatak 3.2

U ovom zadatku potrebno je implementirati izračun vrijednosti broja π pomoću sume reda:

$$\frac{4}{n} \sum_{i=1}^n \frac{1}{1 + \left(\frac{i-0.5}{n} \right)^2}$$

Program kao ulazni parametar prima N , koji predstavlja broj elemenata reda.

Potrebno je implementirati tri verzije:

1. **CUDA/OpenCL kernel** – potrebno je napisati vlastiti CUDA ili OpenCL kernel. Rad eksplicitno raspodijelite među dretvama, pri čemu svaka dretva računa dio sume. Parcijalne sume mogu se konačno zbrojiti na hostu.
2. **Numba @vectorize** – svaka instanca funkcije računa jedan doprinos sumi. Završno zbrajanje može se napraviti izvan vektorizirane funkcije.
3. **Numba @guvectorize** – pojedina instanca funkcije računa parcijalnu sumu nad blokom elemenata. Isprobajte različite veličine blokova, npr. 1024, 2048, 4096 i veće vrijednosti ako hardver dopušta.

Za mjerenje koristite $N = 2^{24}$, ili najveću izvedivu vrijednost za dostupni hardver.

Za sve verzije izmjerite vrijeme izvođenja, ponovite mjerenja više puta i rezultate prikažite u tablici.

Usporedite pristupe s obzirom na vrijeme izvođenja i količinu kontrole koju programer ima nad raspodjelom rada i redukcijom.

Zadatak 3.3

Na temelju zadanog programa koji simulira dinamiku fluida u 2D prostoru (*Computational Fluid Dynamics* – CFD), ostvarite paralelnu inačicu korištenjem platforme CUDA ili OpenCL. Identificirajte dijelove programa koji se mogu učinkovito paralelizirati s obzirom na količinu posla, podatkovnih ovisnosti, obrasca pristupa memoriji. Obavezno kratko zabilježite na temelju čega ste donijeli odluke o načinu paralelizacije.

Za potrebe paralelizacije, možete koristiti pojednostavljenu inačicu programa koja ne stvara izlazne datoteke. Uvjet ispravnosti je istovjetan iznos greške kao i u slijednom programu!

ulazni parametri: 64 1000; konačna greška: 0.00125534

Odredite ubrzanje paralelne inačice programa za zadane parametre.

Potrebni materijali:

- potpuna inačica: https://www.fer.unizg.hr/_download/repository/CFD_V.zip
- pojednostavljena inačica: https://www.fer.unizg.hr/_download/repository/CFD.zip